Db2 for i

Un héritage puissant qui ne demande qu'à évoluer

Andy — Ingénieur de données

SINAPSE

iBelieve 2026 · 15 octobre 2026

iBelieve 2026
Présent et Futur de l'IBM i

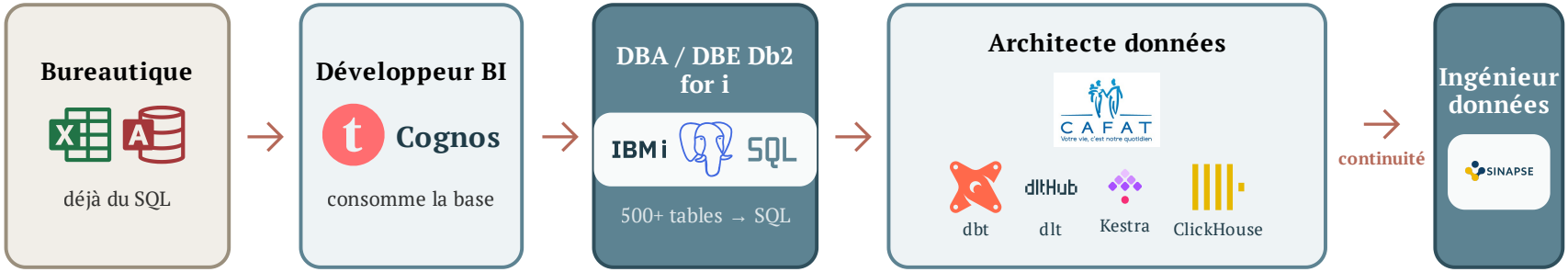
Co-organisé par itheis IBM

Qui suis-je ?

Andy

Ingénieur de données · Sinapse

- iBelieve 2021 : **500+ tables DDS → SQL**
- **Fan assumé de l'IBM i**, depuis mes années DBA
- **Le passeur** : l'existant IBM i ↔ la data moderne



Sinapse : la suite de la CAFAT

GIE · 2024
CAFAT · Gouvernement · FSH

265 000
Calédoniens concernés

Étape par étape
le recouvrement d'abord

Membres fondateurs

CAFAT — Votre vie, c'est notre quotidien
Gouvernement de la Nouvelle-Calédonie
fonds social de l'habitat — Bien plus qu'un toit

→ SINAPSE → étape par étape → **SI social modernisé**

Hier **architecte de données à la CAFAT**, aujourd'hui je **construis la suite**.

sinapse.nc — Qui sommes-nous ? · sinapse.nc — Projet · sinapse.nc

Faire évoluer son IBM i : par où commencer ?

40 ans d'applicatif, des règles dans les programmes, peu de documentation

Avec un outil
jusqu'où aident-ils ?

Sans outil
comment avancer quand même ?

IBM i

Avec le natif seul
que permet le SQL ?

Le fil de la réponse : le SQL, **bien plus que des transactions**

moderniser

administrer

exploiter

ouvrir les données

1. Le chantier 2019-2022

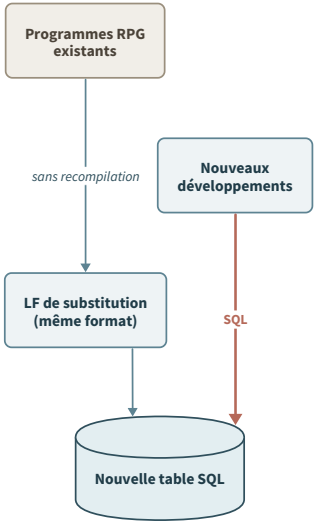
500+

tables DDS → SQL, **sans recompiler**

itheis GAIA + Xcase for i

ET APRÈS

contraintes **temporel** **RCAC**



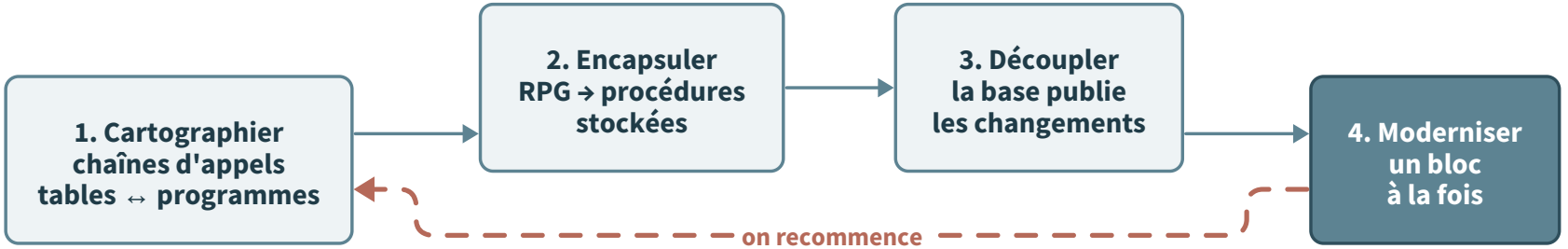
❤ Merci à **E. Sadiman**, qui m'a confié ce chantier — et m'a rendu fan de l'IBM i

> iBelieve 2021 : **migrer des tables**. iBelieve 2026 : en faire **une plateforme de données**.

Source : Itheis — méthode surrogate

2. Moderniser progressivement

On ne réécrit pas 40 ans d'applicatif d'un coup

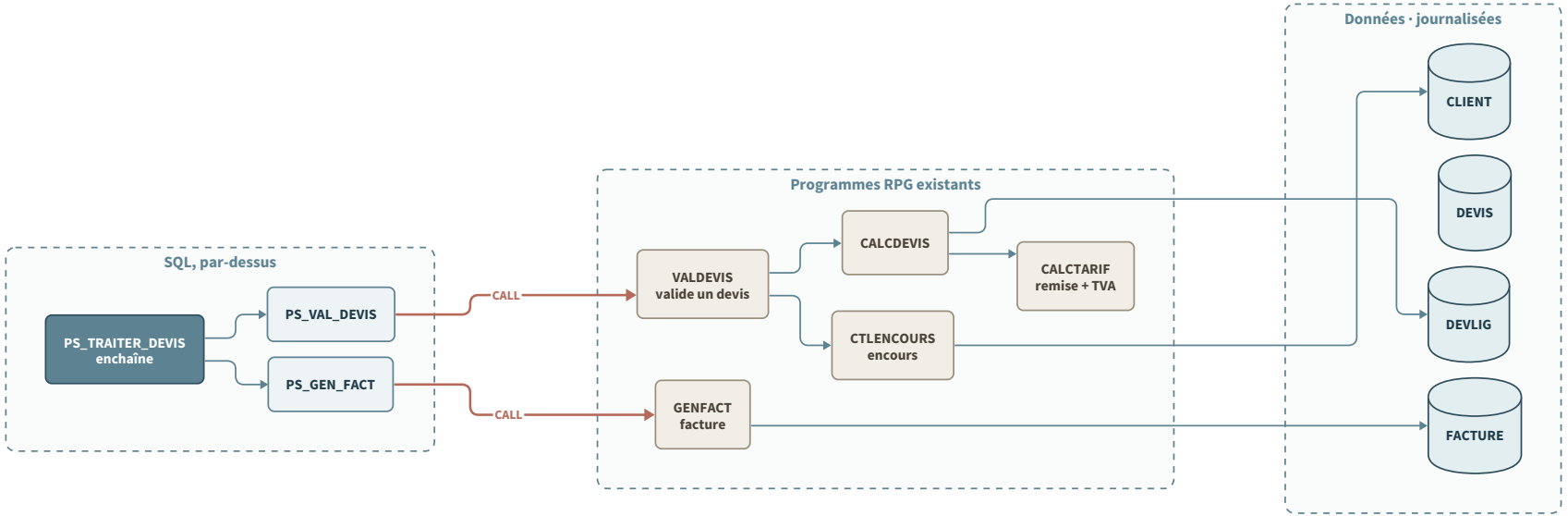


Le SQL est l'outil de chacune de ces étapes.

Principes : SG24-8185 · Redbook 2026 MD260020 · REDP-5095

Le terrain : une mini-application de devis

✓ Tout le SQL qui suit a **tourné** sur pub400 — et se rejoue avec le **kit** du dépôt



Kit : `kit/rpg/` — 5 tables, 7 programmes RPG compilés depuis ACS, 3 procédures SQL · sans compilateur : `kit/00-terrain.sql`

2.1 Cartographier

En clair : savoir qui appelle quoi avant de toucher à quoi que ce soit.



Avec un outil

X-Analysis : modèle déduit du code



Avec l'IA

Bob ou un autre assistant lit le RPG

IBM i

Sans outil

DSPPGMREF, piloté en SQL

```
CALL QSYS2.QCMDEXC(
  'DSPPGMREF PGM(' CONCAT CURRENT SCHEMA CONCAT '/*ALL) OUTPUT(*OUTFILE) OBJTYPE(*PGM) '
  CONCAT 'OUTFILE(' CONCAT CURRENT SCHEMA CONCAT '/T_PGMREF) OUTMBR(*FIRST *REPLACE)');
```

8 programmes analysés → **3** appels entre eux + **9** références de tables

X-Analysis · DSPPGMREF · Format QADSPPGM · IBM Bob pour IBM i · IT Jungle — Bob 1.0 · `kit/rpg/15-cartographier-rpg.sql`

La chaîne d'appels, en deux requêtes

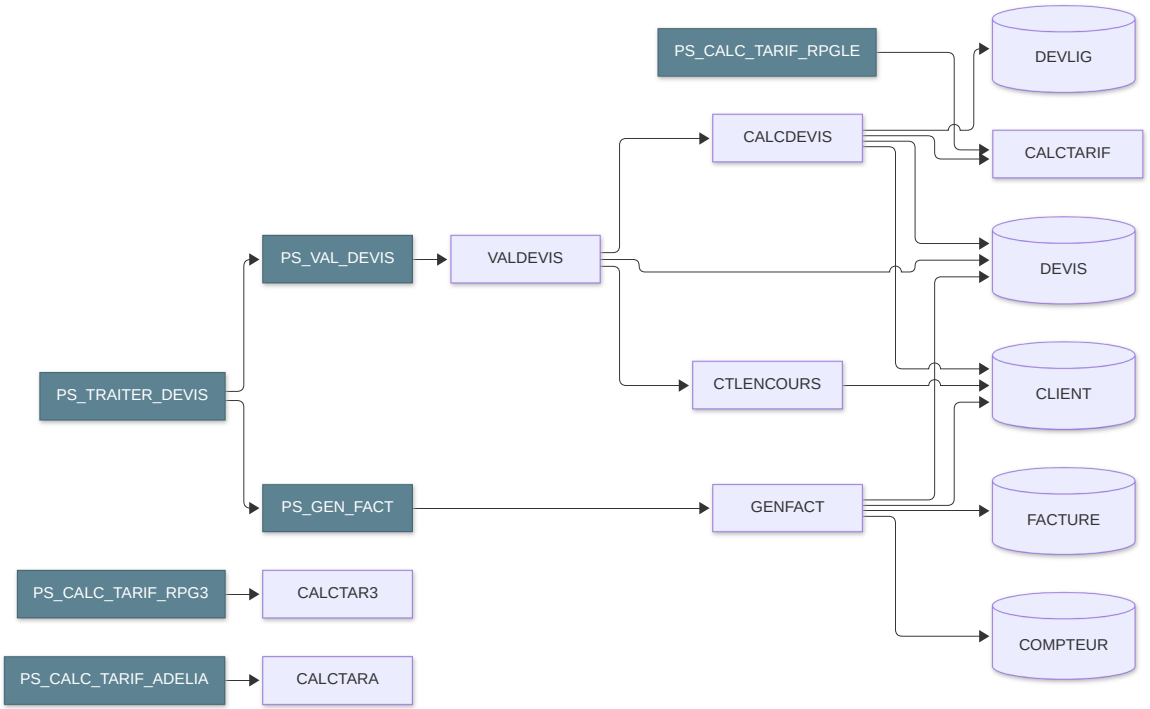
Résultat sur pub400

NIVEAU	CHEMIN	SOURCE
1	PS_TRAITER_DEVIS > PS_GEN_FACT	SYSROUTINEDEP
2	PS_TRAITER_DEVIS > PS_GEN_FACT > GENFACT	SYSROUTINES
1	PS_TRAITER_DEVIS > PS_VAL_DEVIS	SYSROUTINEDEP
2	PS_TRAITER_DEVIS > PS_VAL_DEVIS > VALDEVIS	SYSROUTINES
3	PS_TRAITER_DEVIS > PS_VAL_DEVIS > VALDEVIS > CALCDEVIS	DSPPGMREF
4	PS_TRAITER_DEVIS > PS_VAL_DEVIS > VALDEVIS > CALCDEVIS > CALCTARIF	DSPPGMREF
3	PS_TRAITER_DEVIS > PS_VAL_DEVIS > VALDEVIS > CTLENCOURS	DSPPGMREF

7 ligne(s)

SYSROUTINEDEP · SYSROUTINES · `kit/rpg/15-cartographier-rpg.sql`

Un diagramme sans outil



> Le natif ne remplace pas un outil. Il permet de savoir **par où commencer** et **quoi ne pas casser**.

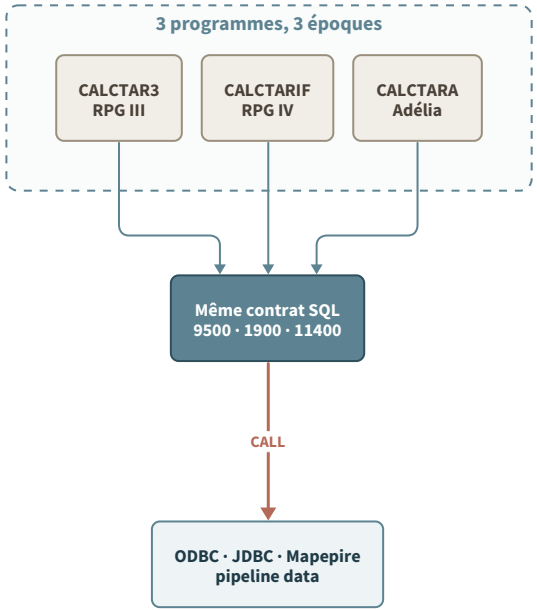
Volubis · IBM i SQL Services · Bob Cozzi — DSPPGMREF en UDTF · `kit/rpg/15-cartographier-rpg.sql`

2.2 Encapsuler

Un programme existant devient une **procédure stockée**. Sans toucher à son code.

```
CREATE OR REPLACE PROCEDURE PS_CALC_TARIF_RPG3 (
    IN  p_montant_ht   DECIMAL(9,2),
    IN  p_taux_remise  DECIMAL(5,2),
    OUT p_remise       DECIMAL(11,2),
    OUT p_tva          DECIMAL(11,2),
    OUT p_ttc          DECIMAL(11,2),
    OUT p_rc           CHAR(1)
)
  LANGUAGE RPG                       -- RPG, pas RPGLE : c'est de l'OPM
  SPECIFIC PS_TARIF3
  NOT DETERMINISTIC
  NO SQL
  EXTERNAL NAME 'VOTREBIB/CALCTAR3'
  PARAMETER STYLE GENERAL;

CALL PS_CALC_TARIF_RPG3(10000.00, 5.00, ?, ?, ?, ?);
```



Appeler la façade depuis l'extérieur

JDBC : les OUT, directement

```
try (CallableStatement cs = cnx.prepareCall(
        "CALL VOTREBIB.PS_CALC_TARIF_RPG3(?, ?, ?, ?, ?, ?)")) {
  cs.setBigDecimal(1, new BigDecimal("10000.00"));  // HT
  cs.setBigDecimal(2, new BigDecimal("5.00"));      // remise %
  for (int i = 3; i <= 5; i++)                      // OUT
    cs.registerOutParameter(i, Types.DECIMAL, 2);
  cs.registerOutParameter(6, Types.CHAR);
  cs.execute();
  System.out.println(cs.getBigDecimal(3) + " / "
      + cs.getBigDecimal(4) + " / " + cs.getBigDecimal(5));
}
```

ODBC : par une fonction table

⚠ **pyodbc** et bien des outils BI **ne lisent pas les paramètres OUT** d'un `CALL`. On crée donc une **UDTF** (fonction table SQL) qui appelle la façade et rend ses valeurs en ligne : `kit/rpg/16-appeler.sql`.

```
SELECT * FROM TABLE(F_TARIF_RPG3(10000.00, 5.00)) T;
```

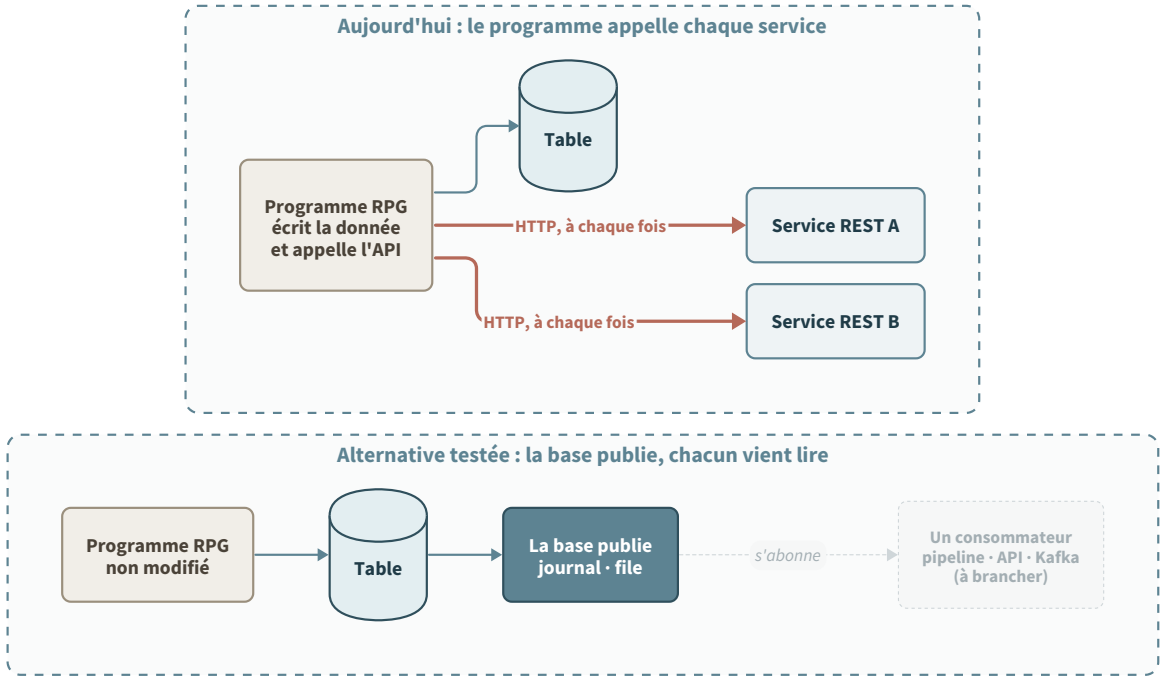
```
import pyodbc

cnx = pyodbc.connect("DSN=MONIBMI")   # IBM i Access ODBC
remise, tva, ttc, rc = cnx.execute(
    "SELECT * FROM TABLE(VOTREBIB.F_TARIF_RPG3(?, ?)) T",
    10000.00, 5.00).fetchone()
print(remise, tva, ttc)   # 9500.00 1900.00 11400.00
```

✓ Dans les deux cas : **9500.00 · 1900.00 · 11400.00**, calculés par le RPG III

2.3 Découpler : publier sans toucher aux programmes

En clair : au lieu que le programme appelle chaque service, la base publie et chacun vient lire.



Le programme n'a pas changé, et **la base publie** ce qui change.

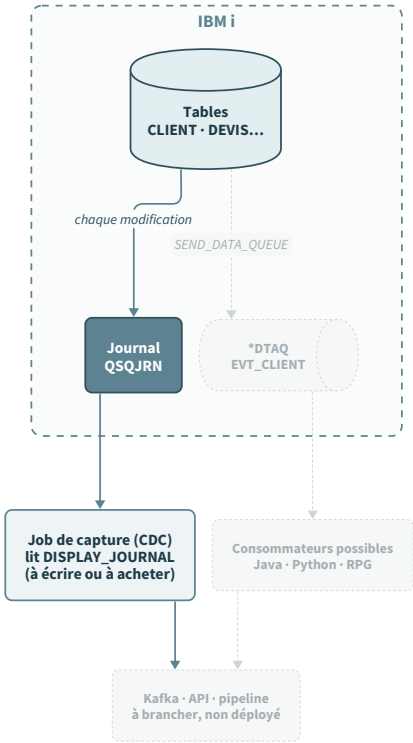
Testé sur pub400 : journal et file. Aucun consommateur n'est déployé.

2.3 Le journal a tout vu

```sql
-- Le journal a déjà tout capté
SELECT ENTRY_TIMESTAMP, JOURNAL_ENTRY_TYPE,
       SUBSTR(OBJECT, 1, 10) AS TABLE_, PROGRAM_NAME
  FROM TABLE (QSYS2.DISPLAY_JOURNAL(
         JOURNAL_LIBRARY    => CURRENT SCHEMA,
         JOURNAL_NAME       => 'QSQJRN',
         STARTING_TIMESTAMP => CURRENT TIMESTAMP - 1 HOUR))
 WHERE JOURNAL_CODE = 'R';
```

```
2026-09-29 09:45:54.211616 | UP | DEVIS    | CALCDEVIS
2026-09-29 09:45:54.2132   | UP | DEVIS    | VALDEVIS
2026-09-29 09:45:54.224304 | UP | COMPTEUR | GENFACT
2026-09-29 09:45:54.23416  | PT | FACTURE  | GENFACT
2026-09-29 09:45:54.235536 | UP | CLIENT   | GENFACT
2026-09-29 09:45:54.236672 | UP | DEVIS    | GENFACT
```

✓ Le journal sait **quel programme** a modifié quoi. Aucun n'a été modifié.



DISPLAY_JOURNAL · `kit/rpg/17-decoupler.sql`

2.3 La base publie

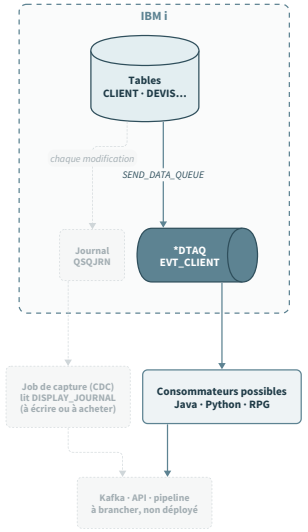
En clair : la base prévient elle-même qui doit l'être, sans modifier les programmes.

```
-- Publier l'état du client en JSON
CALL QSYS2.SEND_DATA_QUEUE_UTF8(
  MESSAGE_DATA => (
    SELECT JSON_OBJECT('table'   VALUE 'CLIENT',
                       'id'      VALUE CLI_NUM,
                       'encours' VALUE CLI_ENCOURS,
                       'ts'      VALUE CURRENT TIMESTAMP)
      FROM CLIENT WHERE CLI_NUM = 1001),
  DATA_QUEUE         => 'EVT_CLIENT',
  DATA_QUEUE_LIBRARY => CURRENT SCHEMA);
```

Ici l'envoi est lancé à la main. Sur un vrai système, un trigger le fait à chaque modification.

```
-- Côté consommateur, y compris via ODBC
SELECT CAST(MESSAGE_DATA_UTF8 AS VARCHAR(1000)) AS MESSAGE
  FROM TABLE (QSYS2.RECEIVE_DATA_QUEUE(
         DATA_QUEUE         => 'EVT_CLIENT',
         DATA_QUEUE_LIBRARY => CURRENT SCHEMA));
```

```
{"table":"CLIENT","id":1001,"encours":2910.00,
 "ts":"2026-09-29-09.45.57.398581"}
```



Java · JTOpen — **Python · ODBC** — **RPG · QRCVDTAQ**

Le SQL sur IBM i n’est pas qu’un langage de requête.

C’est un langage d’**intégration** : il appelle des programmes, publie des événements, parle à MQ et au web.

2.4 Moderniser, un bloc à la fois — puis on recommence sur le suivant.

3. Administrer et exploiter en SQL

En clair : le système se consulte comme une table, sans connaître les commandes.

Une commande CL, un service SQL

Commande CL	Service SQL
`WRKACTJOB`	`QSYS2.ACTIVE_JOB_INFO`
`DSPLOG`	`QSYS2.HISTORY_LOG_INFO`
`DSPJRN`	`QSYS2.DISPLAY_JOURNAL`
`WRKOBJOWN`	`QSYS2.OBJECT_OWNERSHIP`
`WRKSYSVAL`	`QSYS2.SYSTEM_VALUE_INFO`

Les exemples sont livrés



Capture à venir

ACS → Run SQL Scripts → Insert from Examples

`captures/acs-insert-from-examples.png · capture fixe`

Sécurité · exploitation · audit

Audit : qui a quels droits sur CLIENT ?

```
SELECT AUTHORIZATION_NAME, OBJECT_AUTHORITY
  FROM QSYS2.OBJECT_PRIVILEGES
 WHERE SYSTEM_OBJECT_SCHEMA = CURRENT SCHEMA
   AND SYSTEM_OBJECT_NAME   = 'CLIENT';
```

AUTHORIZATION_NAME	OBJECT_AUTHORITY
*PUBLIC	*EXCLUDE
‹propriétaire›	*ALL

2 ligne(s)

SYSTEM_VALUE_INFO · HISTORY_LOG_INFO · OBJECT_PRIVILEGES · Syslog

La méthode



Le système devient **une table comme les autres**

IBM i SQL Services — matérialiser le résultat

4. L'IBM i, source de données de premier plan

En clair : les données de l'IBM i alimentent les outils modernes, sans recopie à la main.



ODBC
Python · BI · DuckDB



JDBC
Java · Talend



Mapepire
un seul port chiffré



HTTP
l'IBM i appelle des API



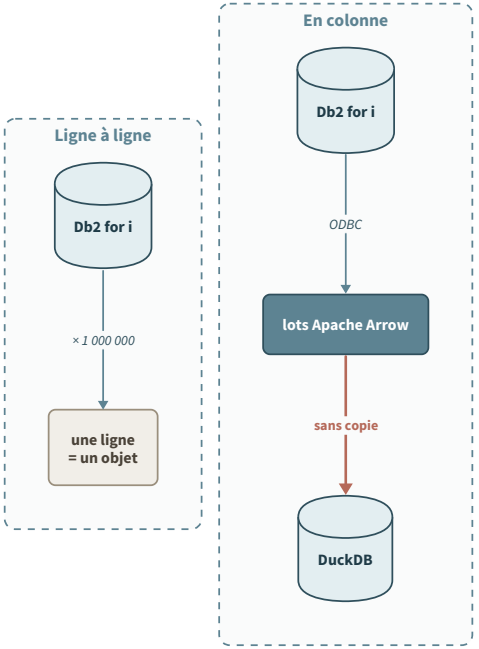
MCP
assistants IA

Les procédures de l'étape 2 passent **par les mêmes portes**

ODBC ACS · ODBC Linux · JTOpen · Mapepire · MCP

ODBC → Arrow → DuckDB

Code illustratif, **non exécuté sur pub400** : le pilote ODBC ne passe pas par le tunnel SOCKS.




```python
from arrow_odbc import read_arrow_batches_from_odbc
import pyarrow as pa
import duckdb

reader = read_arrow_batches_from_odbc(
    query="SELECT * FROM VOTREBIB.CLIENT",
    connection_string="Driver={IBM i Access "
                      "ODBC Driver};System=MONIBMI;...",
    batch_size=50_000,
)
clients = pa.Table.from_batches(
    reader, schema=reader.schema)

duckdb.sql(
  "CREATE TABLE client AS SELECT * FROM clients")
```

arrow-odbc-py · arrow-odbc (Rust) · nanodbc · duckdb/odbc-scanner

Industrialiser l’extraction

dlt + pyarrow
vers Parquet : 20 à 30× plus rapide

Orchestrer
planifier · rejouer · historiser

GARDE-FOUS

- **lecture seule**
- **TLS**
- **typage**
- **hors pics**

Paralléliser par RRN

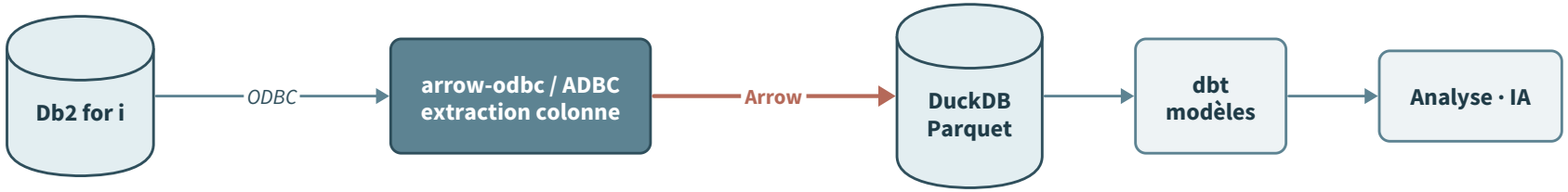
```
-- Taille connue du catalogue
SELECT NUMBER_ROWS
  FROM QSYS2.SYSPARTITIONSTAT
 WHERE TABLE_SCHEMA = CURRENT SCHEMA
   AND TABLE_NAME   = 'DEVLIG';

-- Une tranche parmi N
SELECT * FROM DEVLIG D
 WHERE RRN(D) BETWEEN 1 AND 2;
```

Sur le kit : **4** lignes au catalogue → 2 tranches de 2.

dlt sql_database · ODBC Linux + TLS · Query Supervisor

Sinapse : quand l'outil manque, on le construit



Le besoin

alimenter la plateforme data, sans peser sur la prod

L'existant

ODBC, fiable mais ligne à ligne

Construit

Arrow · ADBC · piste Flight SQL

Résultat

[volumes, temps avant / après]

Apache Arrow ADBC

SINAPSE

À retenir



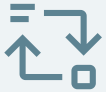
Pas à pas

cartographier · encapsuler · découpler



Le SQL intègre

administre · exploite · publie



Une vraie source

ODBC · Arrow · DuckDB



Un tremplin

l'existant construit la suite

↗ Un moteur qui **évolue** : **17 nouveaux services** à l'été 2026

L’héritage est là.

La seule question, c’est ce qu’on en fait.